

Enhancement Narrative: Software Design and Engineering

Dave Mobley

Southern New Hampshire University

CS-499

Dr. Penmatsa

November 16, 2025

Enhancement Narrative: Software Design and Engineering

1. Artifact Description

Artifact Name: Grazioso Salvare Animal Shelter Dashboard

Origin: CS-340 Advanced Programming Concepts

Creation Date: August 2024 (Module 4-6 Final Submission)

The original artifact was a Jupyter notebook-based dashboard application developed for Grazioso Salvare, a search-and-rescue animal training company. The application used Python with the Dash framework, PyMongo for MongoDB connectivity, and rendered within Jupyter notebooks. It provided basic CRUD operations and data visualization capabilities for the Austin Animal Center database to help identify dogs suitable for rescue training.

The original implementation followed an MVC (Model-View-Controller) pattern within a single notebook file, with MongoDB serving as the data layer, Dash components as the view layer, and Dash callbacks as the controller layer. The dashboard included interactive filters for rescue categories, a sortable data table, a geolocation map, and breed analytics charts.

2. Justification for Inclusion

I selected this artifact for my ePortfolio because it demonstrates a complete data-driven application with real-world utility, but it also clearly showcases opportunities for professional-grade enhancements. The transformation from a proof-of-concept notebook to a production-ready web application demonstrates my ability to:

1. **Architectural Design:** Redesigning a monolithic application into a proper

three-tier architecture (presentation, business logic, data access layers) with clear separation of concerns

2. **API Development:** Creating a RESTful API following industry standards with proper HTTP methods, status codes, and versioning
3. **Security Implementation:** Adding enterprise-grade security features including JWT authentication, role-based authorization, input validation, and comprehensive audit logging
4. **Modern Web Development:** Building a responsive, component-based React frontend with state management and real-time capabilities
5. **DevOps Practices:** Containerizing the application with Docker, implementing CI/CD pipelines, and setting up comprehensive testing infrastructure

The enhanced artifact showcases skills directly relevant to my career goals in technical management and leadership roles in banking/insurance, where security, compliance, and production-ready architecture are critical requirements.

Specific Components Showcasing Skills

Backend API Layer (backend/app/):

- RESTful API design with FastAPI framework
- JWT-based authentication and role-based authorization
- Comprehensive input validation using Pydantic models
- Audit logging service tracking all data operations

- Rate limiting and security headers middleware
- Modular service layer with clear separation of concerns

Frontend Application (`frontend/src/`):

- React application with component-based architecture
- Context API for state management (authentication, data)
- React Router for client-side routing
- API client service with interceptors for error handling
- Responsive design with modern CSS
- Integration with Leaflet maps and ECharts for visualizations

DevOps and Deployment (`deployment/`):

- Docker containerization for backend and frontend
- Docker Compose for local development environment
- GitHub Actions CI/CD pipeline with automated testing
- Multi-stage Docker builds for optimization
- Nginx configuration for production frontend serving

Testing Infrastructure (`backend/tests/`, `frontend/src/test/`):

- Unit tests for API endpoints with mocked dependencies

- Integration tests with real database connections
- Frontend component tests with React Testing Library
- Test coverage reporting and CI integration

How the Artifact Was Improved

The enhancement transformed the application in several key ways:

1. **Architecture:** From monolithic Jupyter notebook to three-tier web application with separate frontend, backend, and database layers
2. **Security:** From no authentication to JWT-based auth with role-based access control and comprehensive audit logging
3. **Scalability:** From client-side data loading (limited to 2000 rows) to server-side pagination and filtering
4. **Deployment:** From Jupyter environment requirement to containerized application deployable anywhere
5. **Maintainability:** From notebook cells to modular, well-documented codebase with comprehensive tests
6. **User Experience:** From notebook interface to modern, responsive web application with professional UI

3. Course Outcomes Achievement

Outcome 2: Professional Communication

Status: Achieved

The enhanced application provides a polished, professional user interface with clear documentation, making complex data accessible to diverse audiences. The API documentation is auto-generated via FastAPI's OpenAPI/Swagger integration, and the codebase includes comprehensive docstrings and comments. The narrative and technical documentation demonstrate my ability to communicate technical concepts clearly.

Outcome 4: Innovative Techniques and Tools

Status: Achieved

The enhancement implements modern web technologies (FastAPI, React, JWT authentication, Docker containerization) that deliver a production-ready solution accomplishing industry-specific goals. The use of FastAPI provides automatic API documentation, async support, and type validation. React enables a modern, component-based frontend with excellent developer experience. Docker containerization ensures consistent deployment across environments.

Outcome 5: Security Mindset

Status: Achieved

The implementation demonstrates a security mindset through:

- JWT authentication with secure token handling
- Role-based authorization (admin vs. user roles)
- Input validation on all endpoints
- Comprehensive audit logging for compliance
- Security headers (CSP, X-Frame-Options, etc.)

- Rate limiting to prevent abuse
- CORS policies for cross-origin security

The audit logging system tracks all data operations with user, timestamp, action details, and IP addresses, providing forensic capabilities essential for regulated industries.

Updates to Coverage Plans

The implementation successfully met all planned outcomes. One addition beyond the original plan was the comprehensive testing infrastructure, which was expanded to include both unit and integration tests for both backend and frontend, ensuring code quality and reliability.

4. Reflection on the Enhancement Process

What I Learned

1. **API Design Best Practices:** Creating RESTful APIs requires careful consideration of HTTP methods, status codes, error handling, and response formats. FastAPI's automatic validation and documentation generation significantly improved development speed and code quality.
2. **Security Implementation:** Implementing JWT authentication taught me about token-based authentication flows, token refresh mechanisms, and secure storage practices. The audit logging system required careful design to capture all necessary information without impacting performance.
3. **State Management:** Managing application state in React using Context API provided insights into when to use global state vs. local state, and how to structure

contexts to avoid unnecessary re-renders.

4. **Containerization:** Docker containerization required understanding multi-stage builds, volume management, and networking between containers. Docker Compose simplified local development significantly.
5. **Testing Strategies:** Writing comprehensive tests required understanding the difference between unit tests (isolated, mocked) and integration tests (real dependencies). Test coverage reporting helped identify gaps in testing.
6. **CI/CD Pipelines:** Setting up GitHub Actions workflows taught me about automated testing, building, and deployment processes. The pipeline ensures code quality before merging.

Challenges Faced

1. **Database Connection Management:** Integrating the original `AnimalShelter` class with the new API architecture required careful handling of database connections. The solution involved creating a service layer that wraps the original class while using the new database manager.
2. **Authentication Flow:** Implementing JWT authentication with proper token refresh and error handling required understanding OAuth2 flows and token lifecycle management.
3. **CORS Configuration:** Configuring CORS correctly for development and production environments required understanding cross-origin request policies and security implications.

4. **State Synchronization:** Keeping the frontend state synchronized with backend data, especially for real-time updates, required careful design of the API client and state management.
5. **Testing Database Operations:** Writing integration tests that work with a real MongoDB instance required setting up test databases and cleaning up test data properly.
6. **Docker Networking:** Configuring Docker Compose networking so that frontend, backend, and MongoDB containers could communicate required understanding Docker network concepts.

Solutions and Approaches

- **Database Integration:** Created a service layer that wraps the original `AnimalShelter` class, allowing reuse of existing business logic while adapting to the new architecture
- **Authentication:** Used FastAPI's `OAuth2PasswordBearer` and `python-jose` for JWT handling, following industry-standard patterns
- **State Management:** Used React Context API for global state (auth, user) and local state for component-specific data
- **Testing:** Separated unit tests (mocked) from integration tests (real DB), using `pytest` fixtures for test setup
- **Containerization:** Used Docker Compose for orchestration, with health checks and dependency management

- **CI/CD:** Created GitHub Actions workflows that run tests, build images, and can be extended for deployment

Professional Growth

This enhancement project significantly advanced my skills in:

- Software architecture and design patterns
- API development and RESTful design
- Security implementation and best practices
- Modern frontend development with React
- DevOps practices including containerization and CI/CD
- Testing strategies and test-driven development

The project demonstrates my ability to take a proof-of-concept and transform it into a production-ready application suitable for enterprise environments, which directly aligns with my career goals in technical management and leadership.